

Lab 4 — Predict a System Through Time

Friday, October 23

Last updated: October 8, 2026.

Date is on the [schedule](#). Same day: [Quiz 7](#) (systems / RREF — **not** dynamics) and a short [Lecture 26](#), then this lab.

Credit: Lab points are for effort in class, not for a Canvas file upload. Work the hub exercises in Octave or Python on a laptop. On lab day you get a **self-contained one-pager** with the exercises written out, space for answers and new syntax, and a blank for instructor initials after a short check-in. Preview the [one-pager](#); write on the hard copy handed out in class. Quizzes will ask about these ideas later; Week 9 dynamics itself is assessed mainly via Midterm 2 / Lab 5, not Quiz 7.

Resources: Lachniet Ch. 2 (matrix ops) · loops / introductory programming in Lachniet · [Syntax sheets](#) · [Resources](#) · Lectures [24–26](#).

What we are doing

Implement a linear dynamical model $x_{t+1} = Ax_t$, iterate many steps, hand-check the first ones, compare a parameter change, and **write down** a long-run pattern you cannot yet explain. That mystery returns in Week 13 (eigenvectors) — do not try to invent the theory today.

Learning objectives

1. Implement a linear dynamical model.
2. Write or complete a short iterative calculation.
3. Validate the first steps by hand.
4. Compare results under altered parameters or initial states.
5. Record an unexplained long-run pattern for later investigation.

Math connections

State and transition matrix ([Lecture 24](#)); simulation, plots, and long-run names without eigenvalues ([Lecture 25](#)); implement / validate / compare ([Lecture 26](#)). Matrix–vector product Ax from Week 7.

Key syntax (this lab)

Task	Octave	Python
Matrix	<code>A = [0.9 0.2; 0.1 0.8]</code>	<code>A = np.array([[0.9, 0.2], [0.1, 0.8]])</code>
Column vector	<code>x = [1000; 0]</code>	<code>x = np.array([1000., 0.])</code>
One step	<code>x = A*x</code>	<code>x = A @ x</code>
Loop	<code>for t = 1:20, x = A*x; end</code>	<code>for t in range(20): x = A @ x</code>
Plot vs time	<code>plot(0:T, X(1,:), 'o-')</code>	<code>plt.plot(range(T+1), X[0], 'o-')</code>

Indexing reminder: Boyd x_1 is 1-based. Octave is 1-based; NumPy is 0-based.

Demo (optional — run once)

Two campus groups: (on-campus, off-campus). One week of flow:

$$A = \begin{bmatrix} 0.85 & 0.10 \\ 0.15 & 0.90 \end{bmatrix}, \quad x_0 = \begin{bmatrix} 800 \\ 200 \end{bmatrix}.$$

Octave:

```
A = [0.85 0.10; 0.15 0.90];  
x = [800; 200];  
x = A*x      % should match hand calculation of x1
```

Python:

```
import numpy as np  
A = np.array([[0.85, 0.10], [0.15, 0.90]])  
x = np.array([800., 200.])  
x = A @ x     # should match hand calculation of x1
```

Expected: $x_1 = (700, 300)$.

Exercises

Pick Octave or Python for the term. Use column vectors for states.

E1. Hand-check one step. Let

$$A = \begin{bmatrix} 0.9 & 0.2 \\ 0.1 & 0.8 \end{bmatrix}, \quad x_0 = \begin{bmatrix} 1000 \\ 0 \end{bmatrix}.$$

- (a) Compute $x_1 = Ax_0$ on paper.
- (b) Check with code.
- (c) Compute x_2 on paper; check with one more $A*x$ / $A @ x$.

Octave: `A = [0.9 0.2; 0.1 0.8]; x = [1000; 0]; x = A*x`

Python: `A = np.array([[0.9, 0.2], [0.1, 0.8]]); x = np.array([1000., 0.]); x = A @ x`

Expected: $x_1 = (900, 100)$; $x_2 = (830, 170)$.

E2. Iterate. Starting from the same A and x_0 as E1, write a loop that applies A twenty times. Print (or store) x_5 and x_{20} .

Octave sketch:

```
A = [0.9 0.2; 0.1 0.8];  
x = [1000; 0];  
for t = 1:20  
    x = A*x;  
    if t == 5 || t == 20  
        disp(x')  
    end  
end
```

Python sketch:

```
A = np.array([[0.9, 0.2], [0.1, 0.8]])
x = np.array([1000., 0.])
for t in range(1, 21):
    x = A @ x
    if t in (5, 20):
        print(t, x)
```

Expected (rounded): $x_5 \approx (722.7, 277.3)$; $x_{20} \approx (666.9, 333.1)$. (Exact fractions not required — match to about 1 person.)

E3. Plot. Modify E2 to store both components at every time $t = 0, \dots, 20$, then plot each component versus t . Do the curves look like growth, decay, oscillation, or stabilization?

Expected answer: stabilization (approach a steady mix near two-thirds / one-third); no oscillation.

E4. What-if parameter. Replace A by

$$A_{\text{new}} = \begin{bmatrix} 0.7 & 0.2 \\ 0.3 & 0.8 \end{bmatrix}$$

(same $x_0 = (1000, 0)$). Recompute x_1 by hand and with code, then simulate to $t = 20$. In one sentence: how does the long-run on-campus share compare to E2?

Expected: $x_1 = (700, 300)$; by $t = 20$ the first component is near 400 (lower on-campus share than E2's ~ 667).

E5. Mystery log (save for Week 13). Let

$$B = \begin{bmatrix} 1.2 & 0 \\ 0 & 0.5 \end{bmatrix}.$$

Simulate from $x_0 = (1, 0)$ and from $y_0 = (0, 1)$ for about 15 steps each (print a few values or sketch).

(a) Describe the two long-run behaviors in plain language.

(b) On the ticket, write **one question** you cannot yet answer with a formula (eigenvalue theory is off-limits today).

Expected: from $(1, 0)$ the state grows; from $(0, 1)$ it decays toward 0. Sample mystery: “Why does the *direction* of the start decide growth vs decay?”

E6 (stretch). Oscillation. Let $C = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$ and $x_0 = (1, 0)$. Compute $x_1, \dots, x_4$ by hand or code. What geometric map is C ? Does $\|x_t\|$ change?

Expected: $x_1 = (0, 1)$, $x_2 = (-1, 0)$, $x_3 = (0, -1)$, $x_4 = (1, 0)$; 90° CCW rotation; norm stays 1.

One-pager

Preview the [Lab 4 ticket](#); write on the hard copy in class.